HANDBOOK

Workstreams

Parallel, disposable worktrees with a durable ledger

> Spin up isolated worktrees for every unit of work, throw them away when they merge, and keep one ledger that always knows where each branch stands.

Version 0.13.0 · July 2026

CONTENTS

The idea

A workstream is one feature split into many small units of work, each living in its own throwaway git worktree.

When a feature is too big for a single branch, you break it into **units** — a slice, a follow-up, a stacked layer. Each unit gets an isolated git worktree and branch of its own, so units never step on each other and can be built, reviewed, and merged independently. The worktree is a pure code checkout: a scratch space you can create, abandon, and recreate at will.

The catch is that a checkout can vanish — dropped, garbage-collected, or replaced — so nothing you care about keeping can live inside it. Every durable datum — a unit's charter of intent, its progress and follow-ups, its history of decisions and restacks, and its identity map from unit to repo and branch — lives in a single **global store** under `~/.local/share/workstreams/`, one directory per workstream, shared across repos. Volatile facts that git and GitHub already own (current branch, base, PR number, status) are never copied in; they are derived live.

```
~/.local/share/workstreams/<ws-id>/
  workstream.md          # metadata only
  units.md               # append-only ledger (unit ↔ repo/branch identity map)
  units/<unit-id>/
    charter.md           # STATIC: why this unit exists — the north star
    progress.md          # MUTABLE current state: Tasks + Follow-ups checklists
    log.md               # APPEND-ONLY: created, dropped, restack, decision, note
```

> **Golden rule.** The worktree holds only code. Drop and recreate it freely — all progress, history, and identity survive in the store.

The workflow at a glance

`ws-board` — glance at status anytime, from any session; read-only.

Units: up front or ad-hoc — decide the split beforehand (ask your brainstorm/spec step to include the unit breakdown; `ws-init` seeds it into the backlog from the design) or start units ad-hoc at any moment with `ws-start`.

Installation

One command installs all the skills — through the Agent Skills CLI or the Claude Code plugin marketplace; the store and its config scaffold themselves on first use.

Workstreams ships as a set of `ws-*` skills in the portable **Agent Skills** format: the workflow commands, the shared contract (`ws`), and this guide (`ws-guide`). Install once and the `/ws-*` commands are available in every repo, because the store is global rather than per-project.

Prerequisites

- **git** with worktree support (2.5+) — every unit is a separate `git worktree`.
- **GitHub CLI** (`gh`), authenticated with `gh auth login` — the default `forge` flavor derives PR state and base from it.
- An agent that supports **Agent Skills** — Claude Code, Cursor, Codex, and others.

Option A — Agent Skills CLI (skills.sh)

Works with any agent on the Agent Skills standard — Claude Code, Cursor, Codex, and 20+ others. Installs all the skills and keeps them updatable.

```
npx skills add caioariede/claude-plugins
npx skills update    # refresh installed skills later
```

Option B — Claude Code plugin marketplace

Claude Code only. Point it at this repo's marketplace and install the plugin.

```
/plugin marketplace add caioariede/claude-plugins
/plugin install workstreams@caioariede
```

Pick one channel. Installing through both registers the skills twice in Claude Code. Either way, verify with a bare `/ws-config` — it confirms the skills resolve and checks your tools, flagging a missing `gh` before your first `ws-init` fails.

State scaffolds itself. The global store at `~/.local/share/workstreams/` and its `flavors.ini` are created on the first `ws-init` — nothing to set up by hand.

First run

Run `/ws-config` once after installing. It detects which tools you have — `wmx`, superpowers, `gh` — and offers to activate the matching flavors for any group you haven't set; accept and you're configured. It also flags an active flavor whose tool is missing. Defaults are `git-worktree` for worktree management, `gh` for the forge, and `none` for spec-driven development; change any of them later with `ws-config set` — see the Flavors section.

```
$ /ws-config
$ /ws-config set spec-driven-development superpowers
```

The workflow commands

The workflow commands cover the full life of a unit of work — from spinning up a container to tearing a branch down, plus deciding what's next and configuring flavors — while every durable fact lives in the global store, never in a worktree.

Golden rule. Worktrees are disposable code checkouts; all durable state lives in the store. Every command loads the shared contract (the `ws` skill) before it acts, and never stores what git or GitHub already owns (branch, base, PR number, status — all derived live).

`ws-init` `<workstream name>`

Use when you need a new workstream and none exists yet — before starting any unit with `ws-start`.

Computes the `ws-id` (`<YYYY-MM-DD>-<slug>`) and creates the store container: `workstream.md` metadata, an empty append-only `units.md` ledger, and the `units/` directory. Sets up the container only — it creates no unit and no worktree.

```
$ /ws-init task templates
```

`ws-start` `<ws-id> <what this unit does> [--base <unit-id|branch>]`

Use when starting a new unit of work in an existing workstream — its own worktree plus ledger entry. Run `ws-init` first if no workstream exists.

Mints `unit-id = <ws-id>:<slug>`, resolves the repo (`--repo` › a `--base` unit's repo › cwd) and the base, then creates the `<slug>` branch and worktree off the base via the active `worktree-management` flavor's `create`. Appends the ledger line and seeds `charter.md` (the unit's static intent), `progress.md`, and `log.md`; `ws-resume` plans and builds.

```
$ /ws-start 2026-07-03-task-templates "add is_template flag" --base schema-migration
```

`ws-resume` `[unit-id]`

Use when resuming or continuing an existing unit, or picking work back up from inside its worktree.

With no argument, infers the unit from the current worktree's branch. Ensures the worktree exists (reopening or recreating it if gone), reconciles the base against the recorded one and realigns on drift, then reads `charter.md` to reconstruct the unit's intent and loads `progress.md` and `log.md` to continue at the first unchecked task.

```
$ /ws-resume add-is-template-flag
```

`ws-board` `[ws-id] [unit-id]`

Use when you want to see or share where a workstream stands — "show the board", "what's done", "workstream status".

Reads the ledger, derives each unit's status and task counts, and renders copy-paste-ready markdown grouped by state (Done, In progress, Blocked, Not started, Dropped) with open follow-ups listed across units. Given a `unit-id`, prints that unit's full checklist and recent notes.

```
$ /ws-board 2026-07-03-task-templates
```

Tip. `ws-board` is strictly read-only — it derives everything and reconciles nothing. Reach for it freely to snapshot progress without touching git or GitHub.

Blocked units. A unit is `blocked` whenever it has an unmet need — its base or a dependency added with `ws-block` — and shows in the board's ⛔ Blocked column until that need clears.

`ws-block` `<unit> needs <target> ["note"] | <unit> clear N<n>`

Use when a unit must wait on another before it can proceed — record, view, or remove a dependency: "B needs A", "B is blocked by A".

Adds or removes a unit's **needs** in its `progress.md` `## Needs` section. A `<target>` is a unit or a follow-up (`<unit-id>:F<n>` / `WF<n>`); it's satisfied once that unit is code-complete (every task checked) or that follow-up is checked. Any unmet need derives the `blocked` status and routes the unit into the board's ⛔ Blocked column. `clear N<n>` removes a need on a genuine scope change — it never marks one satisfied, since satisfaction is always derived.

```
$ /ws-block add-is-template-flag needs schema-migration
```

`ws-backlog`

`"<what>" [--defer | --here | --plan --base <x> [--needs a,b]] [--to <unit>]`

Use when future work surfaces mid-workstream and needs capturing — a bug or follow-up out of scope for the unit you're in, a "we should also do X later" idea, or a whole unit worth planning.

Routes the item to its one correct home so nothing orphans: a unit's `progress.md` `## Follow-ups` (`F<n>`, fixed before this unit's PR merges), the workstream's `backlog.md` `## Follow-ups` (`WF<n>`, deferred), or `backlog.md` `## Planned units` (a future unit). Inside a unit it asks the one thing it can't derive — resolve before this PR merges? — while flags decide it non-interactively. It never writes `## Tasks`: the plan comes from planning, not by hand.

```
$ /ws-backlog "retry backoff is wrong" --defer
```

`ws-restack` `<unit-id> <new-base>`

Use when a unit's base PR merged and its branch must move onto a new base, or GitHub auto-retargeted a dependent PR and the local branch needs realigning.

Resolves `<new-base>` (a branch, or another unit-id → that unit's branch) and rebases per the SPEC reconciliation. When *we* initiate — the PR's `baseRefName` is unchanged remotely — it runs `gh pr edit --base` first; in the auto case it skips that. Stops and reports on conflicts.

```
$ /ws-restack add-is-template-flag master
```

`ws-drop` `<unit-id>`

Use when you want to drop, tear down, abandon, or clean up a unit's worktree and branch.

Shows exactly what will be removed and requires explicit confirmation, then removes the worktree and window and deletes the *local* branch. Appends a `dropped` log line but keeps `progress.md` and the ledger line as the reconciliation record.

```
$ /ws-drop add-is-template-flag
```

Gotcha. `ws-drop` never deletes the remote branch or closes the PR unless you ask, and it preserves the store. To restart, run `ws-start` with the same intent — it versions the id with a `-N` suffix and records `restart-of` automatically.

`ws-next` `[ws-id]`

Use when you're unsure which `ws-*` command or which unit to act on next — after finishing a unit, when a PR merges, or any "what now?" moment.

Reads the ledger and each unit's live status, then lists every unit you could act on right now — ranked, with one marked as the default — and what each needs. It decides, it doesn't do the work (`ws-resume` does). Read-only, like `ws-board`. Nothing starts on its own: once you pick a unit, it asks where that work should run, and an active flavor can take over that question with a flavor hook and present its own choices.

```
$ /ws-next 2026-07-03-task-templates
```

`ws-config`

`[show | set <group> <flavor> | set-overrides <path> | list [group] | add <group> <flavor>]`

Use when viewing or changing workstream flavors — which external tool backs each behavior group.

Edits only `~/.local/share/workstreams/flavors.ini` and the spec-watch script under the store's `hooks/` — never a worktree. `show` (the bare default) prints the active flavor per group and which layers are in play, annotates which flavors' tools are detected, flags an active flavor whose tool is missing, and offers to activate detected flavors for unset groups; `set` switches one group's flavor; `add` scaffolds a custom flavor stub; `set-overrides` points at an external overrides file. Every run also reconciles the spec-watch script to the active flavors. See the Flavors section.

```
$ /ws-config
$ /ws-config set worktree-management wmx
```

Flavors

External tools are pluggable. The skills never hardwire a worktree tool, a planning methodology, or a forge — each is a **flavor** you choose, so the same `ws-*` commands drive whatever you've configured.

A **group** is a fixed behavior category the skills define; a **flavor** is one implementation of it; an **operation** is a named slot a flavor fills with a one-line instruction (a shell command, or a skill to invoke). Exactly one flavor per group is active, globally. Swapping a flavor changes only mechanism — the ws bookkeeping (progress, log, ledger, PR-ready) is intrinsic and never moves.

Group	Operations	Built-in flavors (default first)
`worktree-management`	create · remove · locate	`git-worktree`, `wmx`
`spec-driven-development`	plan · execute · ship · spec-glob	`none`, `superpowers`
`forge`	default-branch · pr-status · pr-create · pr-ready · pr-retarget	`gh`

Your selection and any custom flavors live in `~/.local/share/workstreams/flavors.ini`, merged over the plugin's built-in defs; an optional overrides file layers on top. View or change all of it with `ws-config`.

Configure with `/ws-config`. Bare `/ws-config` shows the active flavor per group, detects installed tools, and offers to activate matching flavors for groups you haven't set; `ws-config set <group> <flavor>` switches one; `ws-config add` scaffolds a custom flavor to fill in. `gh` is the assumed forge baseline — a non-GitHub user adds a custom `forge` flavor via the overrides file.

Customizing

Flavor definitions merge across three layers, lowest to highest precedence — the plugin's built-in `flavors.ini`, your store `~/.local/share/workstreams/flavors.ini`, then an optional external overrides file. Higher layers win **per operation**, so you redefine only the keys you care about and inherit the rest.

1. **Switch the active flavor** — the common case. `ws-config set <group> <flavor>` rewrites the group's line under `[active]` in your store file; nothing else changes.
2. **Customize or add a flavor** — `ws-config add <group> <flavor>` scaffolds a `[<group>/<flavor>]` section to fill in. Redefine only the operations you want to change.

3. **Layer an external file** — `ws-config set-overrides <path>` records `overrides-file=<path>` under `[config]`, so a file outside the store (a shared team config, say) wins over both lower layers.

Each operation's instruction is a `plugin:skill` id (invoked as a skill), a `ws-*` command line (invoked as that skill), or a shell command — with `<branch>`, `<base>`, `<path>`, `<repo>`, `<pr>`, and `<new-base>` filled from context; hook instructions and their prompts/choices may also use `<unit>` and `<command>` (the firing skill's resolved next command). A non-GitHub user, for instance, replaces the `forge` group with a custom flavor:

```
; ~/.local/share/workstreams/flavors.ini
[active]
forge = glab

[forge/glab]
default-branch = glab repo view -F json | jq -r .default_branch
pr-status      = glab mr view <branch> -F json
pr-create      = glab mr create --source-branch <branch> --target-branch <base> --draft
pr-ready       = glab mr update <pr> --ready
pr-retarget    = glab mr update <pr> --target-branch <new-base>
```

Missing keys fall back. An operation no layer defines resolves to the group's **default** flavor (`git-worktree` / `none` / `gh`); an unreadable `overrides-file` is warned and skipped. Define every operation your custom flavor needs — don't lean on a partial section.

Flavor hooks

A **hook** lets a flavor react at a skill's lifecycle point — an optional operation named `hook-<skill>-<event>`. Each skill documents the events it fires; at an event it runs that hook from every active flavor that defines it. Hooks fire **only in an interactive session** — a subagent or headless run fires none, so a hook never steals focus or blocks automation.

Hooks can prompt. Companion keys (valid only on `hook-*` operations) make one interactive: with no `.prompt` the hook just runs; a `.prompt` and no choices is a yes/no (Yes runs the base instruction, No skips); adding `.choices.<name>` entries turns it into a 2–4-option pick, each with an optional `.choices.<name>.desc` (the chosen option's instruction runs, the base is ignored; an empty instruction — and a dismissed prompt — skips).

A hook at a skill's next-step offer is a **chain hook**: its prompt replaces the default "run it now? (default yes)". The skill stays unaware of what the choices mean — a choice resolving to `<command>` runs the named command in this session; any other choice is the flavor handing the work off its own way, so the skill runs it, re-emits the command, and stops.

The built-in `wmx` flavor uses both kinds: it creates worktrees windowless, then at the end of `ws-start` asks whether to open a new window, and at the end of `ws-next` asks where the picked unit should run.

```
; built-in — worktree-management/wmx
create                            = wmx worktree create <branch> --base <base> --no-window
hook-ws-start-after               = wmx window open <branch>
hook-ws-start-after.prompt        = Open <branch> in a new window? (No = keep working
here)
hook-ws-next-after.prompt              = Where should <unit> run?
hook-ws-next-after.choices.skip        =
hook-ws-next-after.choices.skip.desc   = Not now
hook-ws-next-after.choices.here        = <command>
hook-ws-next-after.choices.here.desc   = Here
hook-ws-next-after.choices.window      = wmx window open <branch>
hook-ws-next-after.choices.window.desc = New window
```

Choices appear in the order the flavor file lists them, and the first is the safe one — a dismissal never starts work. An option whose instruction names something the skill can't supply is dropped from the pick: on a `start` move the unit has no branch yet, so "New window" disappears and the pick narrows to not-now-or-here — `ws-start` offers the window itself once the worktree exists.

> **Silence or force it to taste.** In your store `flavors.ini`, set `hook-ws-start-after.prompt =` (empty) to always open without asking, or `hook-ws-start-after =` (empty) to never open — one line each, merged over the built-in. The same levers work on `hook-ws-next-after` and its `.choices.*` keys.

Spec-watch: from spec to workstream

Workstreams usually begin at a design spec — and the moment one is written is easy to miss. **Spec-watch** closes that gap: when a written file matches the active `spec-driven-development` flavor's `spec-glob` and no workstream's `design:` claims it, the runtime nudges the session to offer `ws-init` with that spec as the design. A workstream that has no design yet is surfaced as the attach-instead alternative. It only suggests — single-PR work is left alone, and nothing is created without you.

The mechanism costs nothing when idle: the plugin's file-write hook merely checks whether `~/.local/share/workstreams/hooks/spec-watch-<flavor>.sh` exists and runs it — the installed script *is* the on-switch. `ws-config` keeps it in sync on every run: a flavor that declares `spec-glob` gets the script installed with its glob baked in; a flavor without one gets it removed. Ownership is matched by the spec's **filename** against `design:` lines, so `~`, absolute, and symlinked spellings all count.

> **Opt out in one line.** Set `spec-glob =` (empty) on your active flavor in the store `flavors.ini`, then any `/ws-config` run removes the watcher — the same empty-key convention that silences flavor hooks.

Using with Superpowers

The `superpowers` flavor of `spec-driven-development` wires the ws commands into the Superpowers loop — but it moves where planning happens. On its own, Superpowers runs **brainstorm** → **write-plan** →

execute in one branch. With workstreams the top-level plan step drops out: a feature is many units, and each unit plans itself.

1. **Brainstorm the spec.** Run the Superpowers `brainstorming` skill as usual to produce the design doc.
2. **Call `ws-init` the moment the spec exists** — before writing any plan. With the superpowers flavor active, spec-watch nudges this step automatically the moment the design doc is written. `ws-init` creates the workstream and records the design on `workstream.md`. Do **not** run `writing-plans` here; there is no top-level plan.
3. **Split the spec into units** with `ws-start`, one per shippable slice — each unit's `charter.md` captures that slice's intent against the shared design.
4. **Let each unit plan itself.** `ws-resume` runs `writing-plans` scoped to that unit (tasks land as `T1..` in its `progress.md`), then executes with `subagent-driven-development`.

> **Planning lives in units, not above them.** After the spec, go straight to `ws-init` → `ws-start`. The `writing-plans` step is deferred into each unit's `ws-resume`, so every plan is scoped to one worktree.

Cheat sheet

The workflow commands, one durable store: worktrees hold code, the store holds truth, and status is always derived live from git and GitHub.

Command	Purpose
`ws-init`	Create a new workstream container (metadata + empty ledger). Run before any unit exists.
`ws-start`	Start a unit: its own worktree, branch, ledger line, and `progress.md` / `log.md`.
`ws-resume`	Advance a unit at any stage: plan it, continue its tasks, or ship it — reopening a gone worktree and reconciling base drift as needed.
`ws-board`	Show or share where a workstream stands. Read-only — derives everything, writes nothing.
`ws-block`	Add or remove a unit's needs (dependencies). An unmet need derives the `blocked` status.
`ws-backlog`	Capture future work — routes a follow-up (`F<n>` / `WF<n>`) or a planned unit to its right home so nothing orphans.
`ws-next`	List every unit that can move right now, ranked, and ask where the picked one runs. Read-only — decides, doesn't do.
`ws-restack`	Move a unit's branch onto a new base after the base PR merged or GitHub auto-retargeted.
`ws-drop`	Tear down a unit's worktree and local branch; keep the reconciliation record.
`ws-config`	View or switch flavors — the external tool backing each behavior group. Edits `flavors.ini` only, and reconciles the spec-watch script.

```
~/.local/share/workstreams/<ws-id>/
├── workstream.md          # metadata only (id, name, goal, design)
├── units.md               # append-only ledger: unit ↔ repo/branch identity
└── units/
    └── <unit-id>/
        ├── charter.md      # STATIC: why this unit exists (the north star)
        ├── progress.md     # MUTABLE work-state: Tasks + Follow-ups checklists
        └── log.md          # APPEND-ONLY: created · dropped · restack · decision · note
```

`ws-id` = `<YYYY-MM-DD>-<slug(name)>` · `unit-id` = `<ws-id>:<slug(what)>` (bare `<slug>` shorthand; `-N` = restart) · `branch` = `<slug>`. The store is global, not per repo.

Worktree = code only. Never write store files into a worktree. Find a unit's worktree via its ledger branch; drop and recreate worktrees freely — progress survives in the store.

progress.md = work-state source of truth (mutable). Tasks and follow-ups live here and nowhere else. Ids (`T1` , `F1`) are monotonic per unit and never reused, even after check-off or removal. No history is kept here.

log.md = append-only. One line per event — `created` , `dropped` , `restack` , `decision` , `note` . Never edit or delete prior lines; the log never stores current state.

Nothing volatile is stored — derive it live. Current branch comes from git, base and retargeting from `gh pr view --json baseRefName` , PR number and draft/ready/merged from GitHub, and status from a first-match cascade: dropped → merged → blocked → in-review → building.

Gotchas & FAQ

Worktrees are disposable code checkouts; every durable fact lives in the global store or is derived live from git and GitHub — so most "I lost it" moments are recoverable.

Golden rule. Never store what git or GitHub owns. Branch, base, PR number, and status are always derived live; only tasks, follow-ups, and history are written to the store. If a worktree feels lost, the store still knows the unit.

Question	Answer
Q: I deleted the worktree — is my work lost?	**A:** No. Worktrees are disposable; all durable state lives in `~/.local/share/workstreams/<ws-id>/`. Run `/ws-resume <unit-id>`: if the branch still exists it reopens the worktree; if the branch is also gone it starts fresh off the repo default, using the store's `progress.md` as the restart baseline.
Q: Which unit am I in?	**A:** Run `/ws-resume` with no argument. It infers the unit from the current worktree's branch by scanning every `units.md` ledger in the store, then resolves the `ws-id`, repo, and branch for you.
Q: My base PR merged — how do I move onto the new base?	**A:** Run `/ws-restack <unit-id> <new-base>`. It rebases `--onto` the new base from the recorded merge-base and appends a `restack` line to `log.md`. When you initiate and the PR base is unchanged remotely, it also runs `gh pr edit --base` first.
Q: GitHub auto-retargeted my dependent PR — do I still restack?	**A:** The realign still happens, but the `gh pr edit` step is skipped since GitHub already moved the base. `/ws-resume` also auto-reconciles: if the live `baseRefName` differs from the recorded base, it realigns and records a `restack` line before continuing.
Q: I opened a per-unit window and it didn't auto-start the agent.	**A:** Windows are optional — run `ws-resume` in your current session, or launch `claude` in the window yourself. A window only autostarts the agent when your `worktree-management` flavor is configured to.
Q: Can a workstream span multiple repos?	**A:** Yes. The store is global, not per-repo. Each unit records its own `repo=<org/repo>` (and branch) on its ledger line in `units.md`, so units of one workstream can live in different repositories.

Optional window. A per-unit window is optional — run `ws-resume` in your current session instead. If you do open one and it doesn't start the agent, run `claude` manually (a window only autostarts when the `worktree-management` flavor is configured to).

Tip. Read-only by design: `ws-board` never reconciles a base. Only `ws-restack` (explicit) and `ws-resume` (on detecting drift) move a branch onto a new base.